

Draft

Banafsheh Karimian

September 24, 2022

Reinforcement Learning, although it has plenty of advantages and applications, is time-consuming and sample inefficient in many problems. Learning from Demonstrations, which alone is limited to only reach what it observes, is used together with Reinforcement Learning to improve learning in many applications including but not limited to robot learning and self driving cars. However, in most of the literature, it is assumed that the demonstrations provided are optimal and if not optimal, they are sub optimal or at least are performing what our agent wants to do, with different quality. This is time consuming and costly to obtain in many tasks and requires human effort. In this thesis, we are going to analyze our ideas on how we can use a heterogeneous set of demonstrations that are not directly related to our agent's goal in the mentioned applications.

1 Introduction

Machine Learning methods are from the most used methods in fields such as robot learning, self driving cars, medication, and etc. Using its methods, human kind was able to explore the deep sea [CCM⁺19] and other planets such as mars [Mom21], assign tasks that are risky or dangerous for humans to autonomous agents [BSM14], save forests from fire [RBG⁺19], clean the oceans [AAS⁺21], and in general improve humans life [MHUA19]. Although machine learning methods have improved our life greatly, they are still lacking compared to humans in performing everyday tasks in a real environment and need further improvements.

One of the most important method from machine learning methods is reinforcement learning which is inspired from human and animal learning [Col19]. The goal of the reinforcement learning agent is to maximize its expected reward, which is appropriate for many applications including robot learning [KBP13], game playing [Szi12], self driving cars [SAPY17], among many other. Allowing agents to learn through interacting with the environment [LZY⁺19a], reinforcement Learning is considerably successful in addressing decision-making problems [LZY⁺19a]. However, it still has some disadvantages, including being time-consuming for complex problems, unable to reach the optimal result, sample inefficiency, and dependency to reward shaping.

Inspired from humans and animals, learning from demonstration is used to address the mentioned problems and reduce the search space of reinforcement learning methods [BPMP17]. Learning from demonstration is well studied in the literature both individually and merged with reinforcement learning, but in most of the literature the demonstrations are assumed to be optimal or at least sub-optimal and directly relevant to what our agent wants to achieve. Gathering this sort of demonstrations is costly and time-consuming and in some cases not even possible. Thus, in this thesis we want to check how we can use diverse demonstrations to improve the performance of reinforcement learning.

2 Related work

Demonstrations are used for more than three decades for learning problems, especially in fields such as robotics. At first, remotely controlling robots was used to gather demonstrations [LP83, DL84, LS84, SD85]. This remote controlling was hard and needed expertise in robot remote controlling, thus other methods were proposed such as moving robot body manually or using computer vision [KII91, KII94, KII95].

At first demonstrations were used alone to only produce the exact behaviour of the demonstrators, but this in fact had many disadvantages. One of the disadvantages come from the fact that demonstrations could not cover the whole state space. In addition to that the quality of demonstrations influenced

the performance of agent greatly. The idea to solve these problems was to use other methods such as Reinforcement Learning alongside learning from demonstrations [ACVB09].

[SK02] used a two phased method to merge reinforcement learning with learning from demonstrations. At the first phase a teacher selects the appropriate actions and the reinforcement learning module trained on the selected actions. The second phase selects the actions according to reinforcement learning module and continues to learn. In addition, [TSC11] records the policies using remotely controlling the agent or applying a sub-optimal policy and extracts some rules from the demonstrations. Then the agent decides online whether to use these rules or use its internal reinforcement learning.

Reinforcement learning could also use learning from demonstrations to improve its performance. One of the most important disadvantages of reinforcement learning methods are being sample inefficient and time-consuming. Similar to [TSC11], in [RKG⁺17] reinforcement learning is used together with learning from demonstrations, but with the goal of decreasing the learning time of reinforcement learning. The error of imitation learning is added to the error of reinforcement learning in [NMA⁺18] and when the reinforcement reached the performance of demonstrations this error will no longer be used. [SC18] also used states of demonstrations as the starting states and [BHS⁺15] biases its decision to demonstrations using shaping reward function. In [AN05] appropriate representations are learned from demonstrations and reinforcement learning is used to continue learning with the learned demonstrations. Other methods such as pre-processing using demonstrations [CJDT17] and filling memory with demonstrations [HVP⁺18, VHS⁺17] are used to improve the learning pace of reinforcement learning algorithms.

Another disadvantage of reinforcement learning is being unable to find optimal policy when the state space is large. In [CZIJT11] the states that look irrelevant to the agents goal are extracted from demonstrations to decrease the search space. In addition, sparse reward environment are difficult for reinforcement learning agents to solve, which is addressed in [KJF18] using encouraging the agent to explore states similar to observations. [Tor20] also adds causality to reinforcement learning from demonstrations to solve sparse reward problems.

Another method to solve disadvantages of reinforcement learning is through using hierarchical reinforcement learning, which has its own disadvantages such as needing for the sub goals and options to be defined manually. [CIJT12] used demonstrations to find sub goals and [ZZMI09, KKGB12, FKSG17] use them to find options.

In recent methods, there are few ones who address improving reinforcement learning from non optimal demonstration. [PIES22] uses a method named self imitation learning [OGSL18], which imitates from its past experiences, to imitate demonstrations and shows that it can work well for sub optimal demonstrations. [ZLDZ22] also uses demonstrations but improves it using reinforcement learning to address issues of sub optimal demonstrations. [CA22] assumes demonstrations to have different point of view and tries to use them to improve its learning.

The work that might work for heterogeneous demonstrations are [CCJZ22], [WL22] and [GKL⁺19]. [CCJZ22] assumes that all demonstrations are from the same task but with different quality. The method proposed is to influence demonstrations by their correspondence weights per each state action pair. [WL22] creates an ensemble of demonstrations and adds pairs that have high weight to the ensemble. Finally, [GKL⁺19] breaks demonstrations as sub demonstrations and uses them as new demonstrations. Although it is not mentioned or evaluated in the relative paper, the mentioned methods might work for heterogeneous demonstrations and we need to compare our method with them.

3 Problem Formulation

In this section, we are going to mathematically formulate the problem that we intend to address in our further studies. We consider a Markov decision process (MDP), defined by $\langle S, A, P, R, \gamma, \mu \rangle$, in which $R : S \times A \rightarrow \mathbb{R}$ is the reward function, $P : S \times A \times S \rightarrow [0, 1]$ is the transition function, $\gamma \in (0, 1)$ is the discount factor, μ is initial state distribution, and S and A denote the state space and action space respectively that are defined for discrete and continuous cases as follows:

$$Discrete : S = \{s_i; i \in (0, N)\}, A = \{a_i; i \in (0, M)\}$$

$$Continuous : S = \langle s_1, \dots, s_d \rangle, s_i \in \mathbb{R}, A = \langle a_1, \dots, a_q \rangle, a_i \in \mathbb{R}$$

A policy $\pi : S \times A \rightarrow [0, 1]$ is map from a state to action probabilities and a trajectory $\tau = \langle s_0, s_1, a_1, \dots, s_T, a_T \rangle$, where T denotes terminal state, can be generated from π . Reinforcement

learning algorithms try to find an optimal policy π^* that maximizes the discounted return, i.e.:

$$\pi^* = \operatorname{argmax}_{\pi} \mathbb{E}[\sum \gamma^t R(s_t, a_t)].$$

Our agent has initially access to pairs of heterogeneous demonstrations ($Demo = \{demo_i; i \in (0, D)\}$) collected from multiple agents performing multiple tasks. These demonstrations can be considered as the input of our problem or they can be derived from a policy or a data set of videos, $V = \{v_1, \dots, v_k\}$, where v_i is video of an agent performing a task. We first assume that these demonstrations are from another MDP with isomorphic transition function P , state space and action space, but different reward function. We can further analyse the case that state space and action space is also different.

4 preliminaries

4.1 Generative Adversarial Networks or GANs

GANs use a generator to generate G fake images and a discriminator D that tries to identify fake images from real ones. In other words, the loss function of GANs are as follows:

$$\log(D(x)) + \log(1 - D(G(x)))$$

where $G(x)$ is the output of the generator and $D(x)$ is the discriminator's output given the real image and $D(G(x))$ is that of the discriminator's given fake image.

4.2 Adversarial Inverse Reinforcement Learning or AIRL

AIRL uses an idea similar to GANs [GPAM⁺14]. Similar to GANs, in IRL the real data is the demonstrations and the fake data is the trajectory generated by the policy (or the generator) that maximizes the to be learned reward. The objective of AIRL discriminator is as follows:

$$\psi \leftarrow \operatorname{argmax} E_{\tau \sim p^*}(\log(D_{\psi}(\tau))) + E_{\tau \sim \pi_{\theta}}(\log(1 - D_{\psi}(\tau)))$$

where $\tau \sim p^*$ means a trajectory from the demonstrations and $\tau \sim \pi_{\theta}$ means a trajectory from the policy that maximizes the learned reward at the current step. The policy π_{θ} is updated using the advantage function multiplied by the gradient of the log of the policy:

$$\pi_{\theta'} \leftarrow -A(s, a) * \log(\pi_{\theta}(a|s))$$

We can use $r(s, a, sp) = A(s, a)$ and set the reward as follows:

$$r_{\psi}(s, a, sp) = g(s) + \gamma h(sp) - h(s)$$

Where $h(s)$ is a shaping term to avoid undesired shaping of $g(s)$ that is a function approximator of the reward function.

4.3 Soft Actor Critic

Soft Actor Critic or SAC is an off policy policy gradient method that uses value function approximation to better train the policy [HZAL18]. SAC maximizes long-term rewards in addition to action randomness which the objective function can be formulated as:

$$J_{\pi}(\phi) = \sum E_{(s_t, a_t) \sim \pi_{\phi}}[r(s_t, a_t) + \alpha * H(\pi(.|s_t))]$$

Where H is entropy which is controlled using α . The method has a policy network, that returns mean and variance which the actions are sampled from. The update of the policy network is based on Q value estimations, which is itself another network. The Q value update is also related to value function which is again estimated using another network. The objective function of policy network, Q network and value network are as follows:

$$\begin{aligned} J_V(\phi) &= E_{S_t \sim D}[\frac{1}{2} * (V_{\psi}(s_t) - E_{a_t \sim \pi_{\phi}}[Q_{\theta}(s_t, a_t) - \log \pi_{\phi}(a_t|s_t)])^2] \\ J_Q(\theta) &= E_{(s_t, a_t) \sim D}[\frac{1}{2}(Q_{\theta}(s_t, a_t) - r(s_t, a_t) - \gamma * E_{s_{t+1} \sim p}[V_{\psi}(s_{t+1})])^2] \\ J_{\pi}(\phi) &= \nabla_{\phi} E_{S_t \sim D, a_t \sim \pi_{\phi}}[D_{KL}(\pi_{\phi} || \frac{\exp(Q_{\theta}(s_t, .))}{Z_{\theta}})] \end{aligned}$$

5 Method Ideas

In this section we will introduce our ideas in using heterogeneous demonstrations.

5.1 Extracting common reward and pre train the policy

We can assume that each of the demonstrations, $demo_i$, is from another MDP with a reward function that can be divided in two functions i.e.:

$$\forall i \in (0, D) : R_i(s, a) = F(s, a) + G_i(s, a)$$

Where $F(s, a)$ is common for each demonstration. An example of such reward function is a robotic arm performing different tasks. The robot must avoid collision and minimize the consumed energy in all of the tasks (common function F) but must learn to perform different tasks (G_i). Another example can be a self driving car learning different tasks such as parking the car or driving to a destination. In all these tasks the car must avoid accidents and follow traffic rules.

Our first idea is to assume that our agent's reward follows the same structure. We can then extract $F(s, a)$ from demonstrations and instead of learning from scratch we can train a base policy that maximizes $F(s, a)$. First, we will mathematically check that for what sort of problems starting from this initial policy results in a better learning than starting from a random policy and then we will discuss our idea.

5.1.1 Mathematical Analysis

We define discounted future state distribution as $d^\pi(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t P(s_t = s | \pi)$ and return of policy π as $J_\pi = \frac{1}{1-\gamma} * E_{s \sim d^\pi, a \sim \pi(a|s)}(r(s, a))$. We also define pseudo regret of policy π as $PR_\pi = c \sum_{k=1}^T (J^{\pi^*} - J_{\pi_k})$, where c is a constant, J^{π^*} is the return of the optimal policy and J_{π_k} is the return of policy at the k th episode. In order to compare the influence of starting from the pre trained policy, π' , with starting from the random policy, π , we can subtract their pseudo regrets:

$$\begin{aligned} PR_\pi - PR_{\pi'} &= c \sum_{k=1}^T (J^{\pi^*} - J_{\pi_k}) - c \sum_{k=1}^T (J^{\pi'^*} - J_{\pi'_k}) = c \sum_{k=1}^T (J_{\pi'_k} - J_{\pi_k}) \\ PR_\pi - PR_{\pi'} &= c \sum_{k=1}^T \left(\frac{1}{1-\gamma} * E_{s \sim d^{\pi'}, a \sim \pi'(a|s)}(r(s, a)) - \frac{1}{1-\gamma} * E_{s \sim d^\pi, a \sim \pi(a|s)}(r(s, a)) \right) \\ PR_\pi - PR_{\pi'} &= \frac{1}{1-\gamma} \sum_{k=1}^T (E_{s \sim d^{\pi'}, a \sim \pi'(a|s)}(r(s, a)) - E_{s \sim d^\pi, a \sim \pi(a|s)}(r(s, a))) \end{aligned}$$

We know that $E(x + y) = E(x) + E(y)$ and we assumed that $r(s, a) = f(s, a) + g(s, a)$ so we can further develop the above formula as:

$$\frac{1}{1-\gamma} \sum_{k=1}^T (E_{s \sim d^{\pi'}, a \sim \pi'(a|s)}(f(s, a)) - E_{s \sim d^\pi, a \sim \pi(a|s)}(f(s, a)) - E_{s \sim d^{\pi'}, a \sim \pi'(a|s)}(g(s, a)) + E_{s \sim d^\pi, a \sim \pi(a|s)}(g(s, a)))$$

For $k = 1$, we know that $E_{s \sim d^{\pi'}, a \sim \pi'(a|s)}(f(s, a)) > E_{s \sim d^\pi, a \sim \pi(a|s)}(f(s, a))$ because the policy π' is pre-trained on $f(s, a)$, but we can not conclude such a relationship between $E_{s \sim d^{\pi'}, a \sim \pi'(a|s)}(g(s, a))$ and $E_{s \sim d^\pi, a \sim \pi(a|s)}(g(s, a))$. So, we need to analyze in what cases does $E_{s \sim d^{\pi'}, a \sim \pi'(a|s)}(g(s, a)) > E_{s \sim d^\pi, a \sim \pi(a|s)}(g(s, a))$ holds true for $k = 1$.

According to Jensen's Inequality, we have $E(h(x)) \leq h(E(x))$ if h is concave. If we assume $h(s, a) = f(s, a) + g(s, a)$, we can conclude the following:

$$\begin{aligned} E_{s \sim d^\pi, a \sim \pi(a|s)}(f(s, a) + g(s, a)) &\leq f(E_{s \sim d^\pi, a \sim \pi(a|s)}(s, a)) + g(E_{s \sim d^\pi, a \sim \pi(a|s)}(s, a)) \\ E_{s \sim d^\pi, a \sim \pi(a|s)}(f(s, a)) + E_{s \sim d^\pi, a \sim \pi(a|s)}(g(s, a)) &\leq f(E_{s \sim d^\pi, a \sim \pi(a|s)}(s, a)) + g(E_{s \sim d^\pi, a \sim \pi(a|s)}(s, a)) \end{aligned}$$

We know that for every random trajectory, its return with respect to f is always less than or equal to $E_{s \sim \pi, a \sim \pi(a|s)}(f(s, a))$, thus we can say that $f(E_{s \sim \pi, a \sim \pi(a|s)}(s, a)) \leq E_{s \sim \pi, a \sim \pi(a|s)}(f(s, a))$. If the same goes for g , i.e. $g(E_{s \sim \pi, a \sim \pi(a|s)}(s, a)) \leq E_{s \sim \pi, a \sim \pi(a|s)}(g(s, a))$ we can prove that:

$$f(E_{s \sim \pi, a \sim \pi(a|s)}(s, a)) + g(E_{s \sim \pi, a \sim \pi(a|s)}(s, a)) \leq E_{s \sim \pi, a \sim \pi(a|s)}(f(s, a)) + E_{s \sim \pi, a \sim \pi(a|s)}(g(s, a))$$

$$E_{s \sim \pi, a \sim \pi(a|s)}(f(s, a)) + E_{s \sim \pi, a \sim \pi(a|s)}(g(s, a)) \leq E_{s \sim \pi, a \sim \pi(a|s)}(f(s, a)) + E_{s \sim \pi, a \sim \pi(a|s)}(g(s, a))$$

Thus we mathematically showed the condition that was obvious: g for a trajectory generated from π' must be the same or better than trajectories of a random policy π on average. One case satisfying this condition is, f and g being disentangled [LZY+19b] with respect to each other. That means achieving each goal should be possible without obtaining the other [GS19]. Another case is f and g being rewards of sub tasks or set of sub tasks required to achieve the goal. That means achieving high reward in g requires achieving high reward in f or vice versa.

5.1.2 Idea

We can extract the common reward using Inverse Reinforcement Learning and optimization methods and then pre train the policy on the extracted reward. Although this method is accurate, it needs enough amount of demonstration from each demonstrator and sufficient number of demonstrators.

For this method, we first need to extract the reward $R_i(s, a)$ for each demonstrator D_i and then find F from the resulted R_i . At the end we need to pre train a policy on F . We will discuss the mentioned steps with more details.

As mentioned before, IRL methods extract reward from set of demonstrations. Maximum Entropy IRL [ZMB+08], Guided Cost Learning [FLA16] and Adversarial IRL [FCAL16] are from most used IRL methods. We use Adversarial IRL (AIRL) for its ability to approximate reward function using only state trajectories. We could also use other methods, assuming we have actions in our demonstrations. After extracting R_i for each demonstrator D_i , we have to extract F from them. One way to do so is to average all R_i , but this methods needs a sufficient number of demonstrators, which may not be provided to us. Another method is using optimization methods. The following loss function needs to be minimized for us to obtain f :

$$\sum_{\text{forall demonstrator } i} (|r_i - f - g_i|^2 - D_{KL}(f||g_i))$$

The $|r_i - f - g_i|^2$ insures that the equation $r_i = f + g_i$ holds for every demonstrator, and $D_{KL}(f||g_i)$ insures that f is as disentangled as possible from g_i . When the f is extracted, we can pre-train our model on f .

5.2 Extracting common reward and training initial policy

The other idea is to skip calculating the common reward and use the demonstrators policy using Free Energy Minimization method [OBD+15]. We could assume that the policies we obtained from observations are points on the policy space that can guide us to the optimal policy. As we mentioned before, the free energy of a probability distribution P is defined as:

$$F_\alpha(P(x)) = \sum_{x \in X} P(x)U(x) - \frac{1}{\alpha} \sum_{x \in X} P(x) * \log\left(\frac{P(x)}{Q(x)}\right)$$

which maximizes the expected reward and minimizes the energy cost. In reinforcement learning P can be a representative of our policy and Q can be assumed as a behavioural policy. If we had one demonstrator we could have assumed its policy as Q , but here we have multiple demonstrators.

We could maximize our expected reward similar to SAC using minimizing $D_{KL}(\pi_\phi || \frac{\exp(Q_\theta(s_t, \cdot))}{Z_\theta})$ and define our information cost as the weighted some of our policy to the policies of the demonstrators or $\sum \lambda_i * D_{KL}(\pi_\phi || \pi_i)$. Thus, we change the policy objective function of SAC as follows:

$$J_\pi(\phi) = \nabla_\phi E_{S_t \sim D, a_t \sim \pi_\phi} [D_{KL}(\pi_\phi || \frac{\exp(Q_\theta(s_t, \cdot))}{Z_\theta})] + \sum \lambda_i * D_{KL}(\pi_\phi || \pi_i)$$

As it is shown in Algorithm 1, after rolling out and saving the experience in the common experience buffer, we use the experience to update the demonstrated policies using off policy SAC and use the above formula to update our policy. The problem with above formula is that if the other policies are spread across the policy space, one of the KL distance terms could become infinity and decreasing the weights will not prevent the objective function to become infinity. We could fix this problem using either smoothing the demonstration policies or transforming the demonstrated policies to a space that their are not so far from each other.

Algorithm 1 Proposed Algorithm V1

Require: policy of each demonstrator π_i
 initialize $\alpha_i, \phi, \psi, \hat{\psi}, \theta$, common buffer D
for it in $iterations$ **do**
 Roll out policy π_{in} k times
 add experience to D
 for each policy π_i **do**
 sample experience from D
 update π_i using SAC
 end for
 sample experience from D
 $J_V(\phi) = E_{S_t \sim D} [\frac{1}{2} * (V_\psi(s_t) - E_{a_t \sim \pi_\phi} [Q_\theta(s_t, a_t) - \log \pi_{\phi_i}(a_t|s_t)])^2]$
 $J_Q(\theta) = E_{(s_t, a_t) \sim D} [\frac{1}{2} (Q_\theta(s_t, a_t) - r(s_t, a_t) - \gamma * E_{s_{t+1} \sim p} [V_{\hat{\psi}}(s_{t+1})])^2]$
 $J_\pi(\phi) = \nabla_\phi E_{S_t \sim D, a_t \sim \pi_\phi} [D_{KL}(\pi_\phi || \frac{\exp(Q_\theta(s_t, \cdot))}{Z_\theta}) + \sum \lambda_i * D_{KL}(\pi_\phi || \pi_i)]$
 $\psi = \psi - \lambda_V * \nabla_\psi J_V(\psi)$
 $\theta = \theta - \lambda_Q * \nabla_\theta J_Q(\theta)$
 $\phi = \phi - \lambda_\pi * \nabla_\phi J_\pi(\phi)$
 $\hat{\psi} = \tau * \psi + (1 - \tau) * \hat{\psi}$
end for

References

- [AAS⁺21] JK Adarsh, OS Anush, R Shrivarshan, S Mithulesh Krishnaan, JK Akash, R Arul, and S Angalaeswari. Ocean surface cleaning autonomous robot (oscar) using object classification technique and path planning algorithm. In *Journal of Physics: Conference Series*, volume 2115, page 012021. IOP Publishing, 2021.
- [ACVB09] Brenna D Argall, Sonia Chernova, Manuela Veloso, and Brett Browning. A survey of robot learning from demonstration. *Robotics and autonomous systems*, 57(5):469–483, 2009.
- [AN05] Pieter Abbeel and Andrew Y Ng. Exploration and apprenticeship learning in reinforcement learning. In *Proceedings of the 22nd international conference on Machine learning*, pages 1–8, 2005.
- [BHS⁺15] Tim Brys, Anna Harutyunyan, Halit Bener Suay, Sonia Chernova, Matthew E Taylor, and Ann Nowé. Reinforcement learning from demonstration through shaping. In *Twenty-fourth international joint conference on artificial intelligence*, 2015.
- [BPMP17] Diana Borsa, Bilal Piot, Rémi Munos, and Olivier Pietquin. Observational learning by reinforcement learning. *arXiv preprint arXiv:1706.06617*, 2017.
- [BSM14] Trupti B Bhondve, R Satyanarayan, and Moresh Mukhedkar. Mobile rescue robot for human body detection in rescue operation of disaster. *International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering*, 3(6):9876–9882, 2014.
- [CA22] André Correia and Luís A Alexandre. Contrastive learning from demonstrations. *arXiv preprint arXiv:2201.12813*, 2022.

- [CCJZ22] Zi-Xuan Chen, Xin-Qiang Cai, Yuan Jiang, and Zhi-Hua Zhou. Anomaly guided policy learning from imperfect demonstrations. In *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems*, pages 244–252, 2022.
- [CCM⁺19] Andrea Cannata, Flavio Cannavò, Salvatore Moschella, Stefano Gresta, and Laura Spina. Exploring the link between microseism and sea ice in antarctica by using machine learning. *Scientific reports*, 9(1):1–15, 2019.
- [CIJT12] Luis C Cobo, Charles L Isbell Jr, and Andrea L Thomaz. Automatic task decomposition and state abstraction from demonstration. Georgia Institute of Technology, 2012.
- [CJDT17] Gabriel V Cruz Jr, Yunshu Du, and Matthew E Taylor. Pre-training neural networks with human demonstrations for deep reinforcement learning. *arXiv preprint arXiv:1709.04083*, 2017.
- [Col19] Anne Gabrielle Eva Collins. Reinforcement learning: bringing together computation and cognition. *Current Opinion in Behavioral Sciences*, 29:63–68, 2019.
- [CZIJT11] Luis Carlos Cobo, Peng Zang, Charles Lee Isbell Jr, and Andrea Lockerd Thomaz. Automatic state abstraction from demonstration. In *Twenty-Second International Joint Conference on Artificial Intelligence*, 2011.
- [DL84] Bruno Dufay and Jean-Claude Latombe. An approach to automatic robot programming based on inductive learning. *The International journal of robotics research*, 3(4):3–20, 1984.
- [FCAL16] Chelsea Finn, Paul Christiano, Pieter Abbeel, and Sergey Levine. A connection between generative adversarial networks, inverse reinforcement learning, and energy-based models. *arXiv preprint arXiv:1611.03852*, 2016.
- [FKSG17] Roy Fox, Sanjay Krishnan, Ion Stoica, and Ken Goldberg. Multi-level discovery of deep options. *arXiv preprint arXiv:1703.08294*, 2017.
- [FLA16] Chelsea Finn, Sergey Levine, and Pieter Abbeel. Guided cost learning: Deep inverse optimal control via policy optimization. In *International conference on machine learning*, pages 49–58. PMLR, 2016.
- [GKL⁺19] Abhishek Gupta, Vikash Kumar, Corey Lynch, Sergey Levine, and Karol Hausman. Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning. *arXiv preprint arXiv:1910.11956*, 2019.
- [GPAM⁺14] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014.
- [GS19] Christopher Grimm and Satinder Singh. Learning independently-obtainable reward functions. *ArXiv*, abs/1901.08649, 2019.
- [HVP⁺18] Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Ian Osband, et al. Deep q-learning from demonstrations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- [HZAL18] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. PMLR, 2018.
- [KBP13] Jens Kober, J Andrew Bagnell, and Jan Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013.
- [KI95] Sing Bing Kang and Katsushi Ikeuchi. A robot system that observes and replicates grasping tasks. In *Proceedings of IEEE International Conference on Computer Vision*, pages 1093–1099. IEEE, 1995.

- [KII91] Yasuo KUNIYOSHI, Hirochika INOUE, and Masayuki INABA. Teaching by showing: generating robot command sequences based on real time visual recognition of human pick and place actions. *Journal of the Robotics Society of Japan*, 9(3):295–303, 1991.
- [KII94] Yasuo Kuniyoshi, Masayuki Inaba, and Hirochika Inoue. Learning by watching: Extracting reusable task knowledge from visual observation of human performance. *IEEE transactions on robotics and automation*, 10(6):799–822, 1994.
- [KJF18] Bingyi Kang, Zequn Jie, and Jiashi Feng. Policy optimization with demonstrations. In *International conference on machine learning*, pages 2469–2478. PMLR, 2018.
- [KKGB12] George Konidaris, Scott Kuindersma, Roderic Grupen, and Andrew Barto. Robot learning from demonstration by constructing skill trees. *The International Journal of Robotics Research*, 31(3):360–375, 2012.
- [LP83] Tomas Lozano-Perez. Robot programming. *Proceedings of the IEEE*, 71(7):821–841, 1983.
- [LS84] Anthony Levas and Mallory Selfridge. A user-friendly high-level robot teaching system. In *Proceedings. 1984 IEEE International Conference on Robotics and Automation*, volume 1, pages 413–416. IEEE, 1984.
- [LZY⁺19a] Zichuan Lin, Li Zhao, Derek Yang, Tao Qin, Tie-Yan Liu, and Guangwen Yang. Distributional reward decomposition for reinforcement learning. *Advances in neural information processing systems*, 32, 2019.
- [LZY⁺19b] Zichuan Lin, Li Zhao, Derek Yang, Tao Qin, Guangwen Yang, and Tie-Yan Liu. *Distributional Reward Decomposition for Reinforcement Learning*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- [MHUA19] Hafiz Suliman Munawar, Ahmad Hammad, Fahim Ullah, and Tauha Hussain Ali. After the flood: A novel application of image processing and machine learning for post-flood disaster management. In *Proceedings of the 2nd International Conference on Sustainable Development in Civil Engineering (ICSDC 2019), Jamshoro, Pakistan*, pages 5–7, 2019.
- [Mom21] Ali Momennasab. Machine learning for mars exploration. *arXiv preprint arXiv:2111.11537*, 2021.
- [NMA⁺18] Ashvin Nair, Bob McGrew, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Overcoming exploration in reinforcement learning with demonstrations. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 6292–6299. IEEE, 2018.
- [OBD⁺15] Pedro A. Ortega, Daniel A. Braun, Justin Dyer, Kee-Eung Kim, and Naftali Tishby. Information-theoretic bounded rationality. *ArXiv*, abs/1512.06789, 2015.
- [OGSL18] Junhyuk Oh, Yijie Guo, Satinder Singh, and Honglak Lee. Self-imitation learning. In *International Conference on Machine Learning*, pages 3878–3887. PMLR, 2018.
- [PIES22] Georgiy Pshikhachev, Dmitry Ivanov, Vladimir Egorov, and Aleksei Shpilman. Self-imitation learning from demonstrations. *arXiv preprint arXiv:2203.10905*, 2022.
- [RBG⁺19] Melonie Richey, David Blum, Meredith Gregory, Kendall Ocasio, Zachary Mostowsky, and Jordan Chandler. Machine learning classification and segmentation of forest fires in wide area motion imagery. In *Proceedings of the Future Technologies Conference*, pages 100–111. Springer, 2019.
- [RKG⁺17] Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel Todorov, and Sergey Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. *arXiv preprint arXiv:1709.10087*, 2017.
- [SAPY17] Ahmad EL Sallab, Mohammed Abdou, Etienne Perot, and Senthil Yogamani. Deep reinforcement learning framework for autonomous driving. *Electronic Imaging*, 29(19):70–76, jan 2017.

- [SC18] Tim Salimans and Richard Chen. Learning montezuma’s revenge from a single demonstration. *arXiv preprint arXiv:1812.03381*, 2018.
- [SD85] Alberto Segre and Gerald DeJong. Explanation-based manipulator learning: Acquisition of planning ability through observation. In *Proceedings. 1985 IEEE International Conference on Robotics and Automation*, volume 2, pages 555–560. IEEE, 1985.
- [SK02] William D Smart and L Pack Kaelbling. Effective reinforcement learning for mobile robots. In *Proceedings 2002 IEEE International Conference on Robotics and Automation (Cat. No. 02CH37292)*, volume 4, pages 3404–3410. IEEE, 2002.
- [Szi12] István Szita. Reinforcement learning in games. In *Reinforcement learning*, pages 539–577. Springer, 2012.
- [Tor20] Lisa Torrey. Reinforcement learning via reasoning from demonstration. *arXiv preprint arXiv:2004.05512*, 2020.
- [TSC11] Matthew E Taylor, Halit Bener Suay, and Sonia Chernova. Integrating reinforcement learning with human demonstrations of varying ability. In *The 10th International Conference on Autonomous Agents and Multiagent Systems-Volume 2*, pages 617–624. Citeseer, 2011.
- [VHS⁺17] Mel Vecerik, Todd Hester, Jonathan Scholz, Fumin Wang, Olivier Pietquin, Bilal Piot, Nicolas Heess, Thomas Rothörl, Thomas Lampe, and Martin Riedmiller. Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards. *arXiv preprint arXiv:1707.08817*, 2017.
- [WL22] Yu Wang and Fang Liu. Efficient reinforcement learning from demonstration using local ensemble and reparameterization with split and merge of expert policies. *arXiv preprint arXiv:2205.11019*, 2022.
- [ZLDZ22] Zhuangdi Zhu, Kaixiang Lin, Bo Dai, and Jiayu Zhou. Self-adaptive imitation learning: Learning tasks with delayed rewards from sub-optimal demonstrations. 2022.
- [ZMB⁺08] Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, Anind K Dey, et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pages 1433–1438. Chicago, IL, USA, 2008.
- [ZZMI09] Peng Zang, Peng Zhou, David Minnen, and Charles Isbell. Discovering options from example trajectories. In *Proceedings of the 26th annual international conference on machine learning*, pages 1217–1224, 2009.